

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Knowledge Representation Design
Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

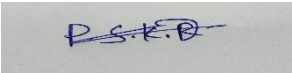
No. of Questions: 5

Annexure A
Knowledge Representation Design Assessment

Code of Conduct:

I P Surendra Kumar Reddy (192425224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

2. Intelligent Loan Approval System

A bank wants to develop an AI system to determine whether a customer is eligible for a loan. The knowledge base contains information about income, credit score, employment, existing loans, and repayment history.

For example:

- Arun has a stable job.
- Arun has a high credit score.
- Arun has sufficient income.
- Customers with stable employment and sufficient income are eligible for preliminary approval.
- Customers with high credit scores and good repayment history are considered low-risk.
- Low-risk customers with preliminary approval can be recommended for loan approval.

Task:

1. Identify the **entities, facts, predicates, and rules** involved.
2. Represent the knowledge using **First-Order Logic**.
3. Build a knowledge base with at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.

5. Apply **forward chaining** to determine whether Arun can be recommended for approval.
6. Apply **backward chaining** for the query `LoanApproved(Arun)`.
7. Use **resolution** to prove or disprove one query.
8. Discuss how the knowledge-engineering process helps maintain the loan rules.

3. Intelligent Vehicle Fault Diagnosis

An automobile service center wants to develop an expert system that identifies possible vehicle faults based on symptoms.

The system knows that:

- A vehicle has a dead battery if it does not start and the headlights are dim.
- A vehicle may have a fuel problem if the engine cranks but does not start.
- A vehicle may have an overheating problem if the temperature is high and coolant is low.
- A vehicle should be sent for immediate service if overheating is detected.

For a vehicle named **Car101**, the system receives:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

Task:

1. Design the **knowledge representation** for the vehicle diagnosis system.
2. Represent at least five rules using **FOL**.
3. Represent selected statements using **Propositional Logic**.
4. Demonstrate **unification** using suitable predicates.
5. Apply **forward chaining** to identify possible faults.
6. Apply **backward chaining** for the query `NeedsService(Car101)`.
7. Demonstrate **resolution** for one diagnosis.
8. Explain how inference is used to arrive at the final diagnosis.

4. Intelligent Student Academic Advising System

A university wants an AI-based academic advising system that recommends actions to students based on their academic performance.

The system contains rules such as:

- Students with attendance below 75% require attendance counseling.
- Students who fail two or more courses require academic counseling.
- Students with high attendance and good academic performance can be recommended for advanced courses.
- Students who have completed prerequisite courses can register for the corresponding advanced course.

Consider a student **Priya** who has:

- Attendance of 68%.
- Two failed courses.
- Completed the prerequisite for an advanced AI course.

Task:

1. Identify the **knowledge engineering requirements** for the system.
2. Represent the academic rules using **FOL**.
3. Create a knowledge base containing facts and rules.
4. Demonstrate **unification** using student and course predicates.
5. Use **forward chaining** to identify recommendations for Priya.
6. Use **backward chaining** to determine whether Priya requires academic counseling.
7. Apply **resolution** to prove one selected recommendation.
8. Explain the advantages and limitations of using a rule-based inference system for academic advising.

5. Cybersecurity Threat Detection System

An organization wants to develop an AI knowledge-based system to identify suspicious activities in its network.

The knowledge base contains rules such as:

- Multiple failed login attempts from the same user may indicate a brute-force attack.
- A login from an unusual location combined with multiple failed attempts indicates suspicious activity.
- Suspicious activity followed by unauthorized file access indicates a possible security breach.
- A possible security breach should generate a high-priority alert.

For user **User101**, the system observes:

- Five failed login attempts.
- Login from an unusual location.
- Unauthorized access to a confidential file.

Task:

1. Design a **knowledge representation model** for the cybersecurity domain.
2. Represent the knowledge using **Propositional Logic and FOL**.
3. Construct at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether a security alert should be generated.
6. Apply **backward chaining** for the query `HighPriorityAlert(User101)`.
7. Use **resolution** to prove the existence of a possible security breach.
8. Compare **forward chaining and backward chaining** for cybersecurity reasoning and justify which is more suitable.

1. Smart Healthcare Diagnosis System Using Knowledge Representation and Reasoning

ABSTRACT

Artificial Intelligence and Expert Systems can be used in healthcare to support preliminary diagnosis by representing medical knowledge as facts and rules and applying logical inference. This report presents the design of a **Smart Healthcare Diagnosis System** using knowledge representation techniques.

The system considers a patient named **Ravi** with fever, cough, and body pain. The knowledge base contains rules stating that fever and cough may indicate flu, flu combined with body pain may indicate viral infection, and patients with viral infection should be advised to rest.

The proposed system represents the knowledge using **Propositional Logic and First-Order Logic (FOL)**. It also demonstrates **knowledge-base construction, unification, forward chaining, backward chaining, and resolution**. The reasoning process shows how the system derives new knowledge from existing facts and ultimately recommends rest for Ravi.

The report demonstrates how a rule-based AIES system can transform medical observations into logical conclusions through a structured inference process.

Keywords: Artificial Intelligence, Expert System, Knowledge Representation, Propositional Logic, First-Order Logic, Unification, Resolution, Forward Chaining, Backward Chaining, Healthcare Diagnosis.

1. INTRODUCTION

Artificial Intelligence enables computer systems to perform tasks that normally require human intelligence, such as reasoning, decision-making, problem solving, and knowledge-based diagnosis.

An **Expert System** is an AI system that stores domain-specific knowledge in a knowledge base and uses an inference engine to derive conclusions. In healthcare, an expert system can represent symptoms as facts and medical relationships as rules.

The given assessment considers a hospital system that identifies possible illnesses from patient symptoms. The knowledge supplied in the problem states that Ravi has fever, cough, and body pain. The system also contains rules connecting these symptoms to flu, viral infection, and the recommendation to rest.

This project demonstrates how these statements can be represented formally and processed using different logical reasoning techniques.

2. PROBLEM STATEMENT

A hospital wants to develop an AI-based system that can identify possible illnesses based on patient symptoms.

The system is given the following information:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

The objective is to develop a knowledge representation model and demonstrate how logical inference can be used to derive Ravi's condition.

3. OBJECTIVES

The main objectives of the proposed Smart Healthcare Diagnosis System are:

1. To identify the entities, facts, predicates, and rules in the healthcare domain.
2. To represent the given knowledge using Propositional Logic.
3. To represent the knowledge using First-Order Logic.
4. To construct a structured knowledge base.
5. To demonstrate unification and substitution.
6. To apply forward chaining to determine Ravi's condition.
7. To apply backward chaining to determine whether Ravi needs rest.
8. To demonstrate resolution for a selected query.
9. To explain the final inference produced by the system.
10. To show how a knowledge-based expert system supports logical decision-making.

4. DOMAIN ANALYSIS

4.1 Domain

The selected domain is **healthcare diagnosis**.

4.2 Main Entity

The primary entity is:

Patient: Ravi

4.3 Observed Symptoms

The system receives three observations:

- Fever
- Cough
- Body pain

4.4 Possible Conditions

The inference process identifies:

- Flu
- Viral infection

4.5 Recommended Action

If viral infection is inferred, the system recommends:

Rest

5. KNOWLEDGE ENGINEERING

Knowledge engineering involves collecting domain knowledge and representing it in a form that an AI system can process.

For this system, the knowledge is divided into:

Facts

Facts describe information that is currently known.

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.

Rules

Rules describe relationships between conditions.

- Fever AND cough $\rightarrow$ Flu
- Flu AND body pain $\rightarrow$ Viral infection
- Viral infection $\rightarrow$ Rest

The knowledge base is structured using facts, predicates, entities, and rules so that the inference engine can process the available patient information.

6. KNOWLEDGE REPRESENTATION

6.1 Entities

Entity	Description
Ravi	Patient
Fever	Symptom
Cough	Symptom
Body Pain	Symptom
Flu	Possible illness
Viral Infection	Possible illness
Rest	Recommended action

6.2 Predicates

The following predicates are used:

- Fever(x)
- Cough(x)
- BodyPain(x)
- Flu(x)
- ViralInfection(x)
- Rest(x)

Here, x represents a patient.

7. PROPOSITIONAL LOGIC REPRESENTATION

Propositional Logic represents statements as propositions that are either true or false.

Let:

- **F** = Ravi has fever.
- **C** = Ravi has cough.
- **B** = Ravi has body pain.
- **L** = Ravi has flu.
- **V** = Ravi has viral infection.
- **R** = Ravi should be advised to rest.

7.1 Facts

The known facts are:

F

C

B

7.2 Rules

Rule 1

If Ravi has fever and cough, Ravi may have flu.

$$F \wedge C \rightarrow L$$

Rule 2

If Ravi has flu and body pain, Ravi may have viral infection.

$$L \wedge B \rightarrow V$$

Rule 3

If Ravi has viral infection, Ravi should be advised to rest.

$$V \rightarrow R$$

Therefore, the complete propositional representation is:

$$F \wedge C \wedge B \wedge (F \wedge C \rightarrow L) \wedge (L \wedge B \rightarrow V) \wedge (V \rightarrow R)$$

8. FIRST-ORDER LOGIC REPRESENTATION

First-Order Logic is more expressive than propositional logic because it represents objects, properties, relationships, variables, and quantifiers.

Let:

- Fever(x) = x has fever
- Cough(x) = x has cough
- BodyPain(x) = x has body pain
- Flu(x) = x has flu
- ViralInfection(x) = x has viral infection
- Rest(x) = x should be advised to rest

The constant **Ravi** represents the patient.

8.1 FOL Facts

1. Fever(Ravi)
2. Cough(Ravi)
3. BodyPain(Ravi)

8.2 FOL Rules

Rule 1

For every patient, if the patient has fever and cough, the patient may have flu.

$$\forall x [(Fever(x) \wedge Cough(x)) \rightarrow Flu(x)]$$

Rule 2

For every patient, if the patient has flu and body pain, the patient may have viral infection.

$$\forall x [(Flu(x) \wedge BodyPain(x)) \rightarrow ViralInfection(x)]$$

Rule 3

For every patient, if the patient has viral infection, the patient should be advised to rest.

$$\forall x [ViralInfection(x) \rightarrow Rest(x)]$$

9. KNOWLEDGE BASE DESIGN

The knowledge base consists of two major components:

9.1 Facts

ID	Fact
F1	Fever(Ravi)
F2	Cough(Ravi)
F3	BodyPain(Ravi)

9.2 Rules

ID	Rule
R1	$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$
R2	$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$
R3	$\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

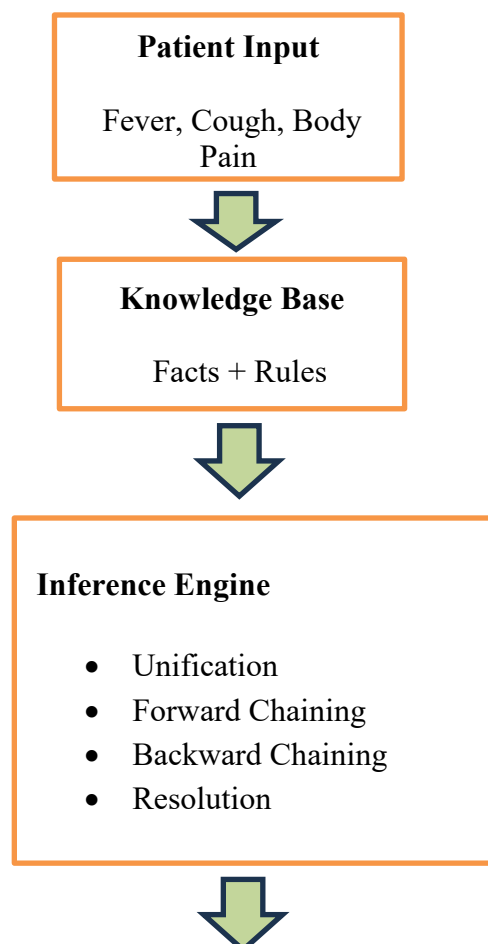
Therefore:

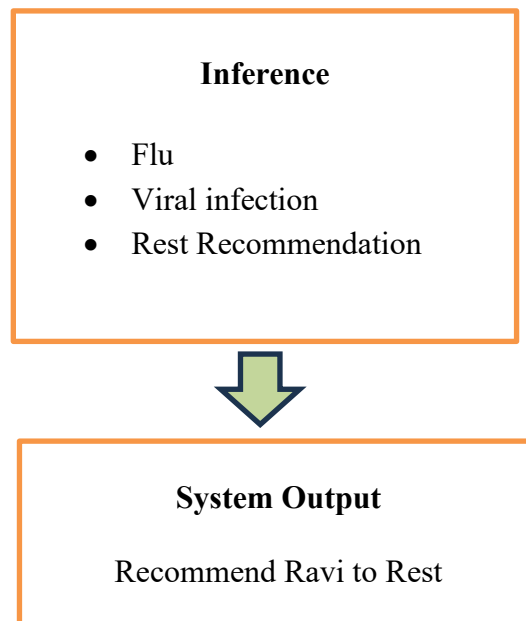
KB = {F1, F2, F3, R1, R2, R3}

The knowledge base is consistent because the given facts do not contradict each other and each derived conclusion follows from an applicable rule.

10. KNOWLEDGE REPRESENTATION ARCHITECTURE

The proposed system follows a simple rule-based expert-system architecture.





The architecture shows how patient observations are processed through the knowledge base and inference engine to produce a logical recommendation.

11. UNIFICATION AND SUBSTITUTION

Unification is the process of making two logical expressions identical by finding suitable substitutions for variables.

11.1 Example 1

Consider:

Fever(x)

and

Fever(Ravi)

The required substitution is:

$\{x/Ravi\}$

After substitution:

$Fever(Ravi) = Fever(Ravi)$

Therefore, the two predicates successfully unify.

MGU

MGU = $\{x/Ravi\}$

11.2 Example 2

Consider:

Cough(x)

and

Cough(Ravi)

Substitution:

$\{x/Ravi\}$

After substitution:

$Cough(Ravi) = Cough(Ravi)$

Therefore, the predicates successfully unify.

MGU

MGU = $\{x/Ravi\}$

11.3 Unification in Rule Application

Consider Rule R1:

$Fever(x) \wedge Cough(x) \rightarrow Flu(x)$

Known facts:

Fever(Ravi)

Cough(Ravi)

Unification gives:

$x = Ravi$

Therefore, Rule R1 becomes:

$Fever(Ravi) \wedge Cough(Ravi) \rightarrow Flu(Ravi)$

Since both premises are true, the system derives:

Flu(Ravi)

12. FORWARD CHAINING

Forward chaining is a **data-driven inference technique**. It starts from known facts and repeatedly applies rules to derive new facts.

12.1 Initial Facts

The system initially knows:

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

12.2 Step 1 – Apply Rule R1

Rule:

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

Substitute:

$x = \text{Ravi}$

Therefore:

$\text{Fever}(\text{Ravi}) \wedge \text{Cough}(\text{Ravi}) \rightarrow \text{Flu}(\text{Ravi})$

Both conditions are satisfied.

New Fact

Flu(Ravi)

12.3 Step 2 – Apply Rule R2

Rule:

$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$

We know:

Flu(Ravi)

and

BodyPain(Ravi)

Therefore:

ViralInfection(Ravi)

12.4 Step 3 – Apply Rule R3

Rule:

$\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

We know:

ViralInfection(Ravi)

Therefore:

Rest(Ravi)

12.5 Forward Chaining Table

Step	Known Information	Rule Applied	New Inference
0	Fever(Ravi), Cough(Ravi), BodyPain(Ravi)	—	Initial facts
1	Fever + Cough	R1	Flu(Ravi)
2	Flu + BodyPain	R2	ViralInfection(Ravi)
3	ViralInfection	R3	Rest(Ravi)

Forward Chaining Result

Ravi → Flu → Viral Infection → Rest

Thus, the forward-chaining process concludes that Ravi should be advised to rest.

13. BACKWARD CHAINING

Backward chaining is a **goal-driven inference technique**. It begins with a desired conclusion and works backward to determine whether the required facts can be established.

Query

Does Ravi need rest?

Goal:

Rest(Ravi)

Step 1

Find a rule that can conclude Rest(x).

Rule R3:

$\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

Therefore, the new sub-goal is:

ViralInfection(Ravi)

Step 2

Find a rule that can conclude ViralInfection(x).

Rule R2:

$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$

Therefore, two sub-goals are created:

Flu(Ravi)

and

BodyPain(Ravi)

BodyPain(Ravi) is already a known fact.

Step 3

To prove Flu(Ravi), use R1:

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

Therefore, we need:

Fever(Ravi)

and

Cough(Ravi)

Both are known facts.

Therefore:

Flu(Ravi) is proved.

13.1 Backward-Chaining Tree

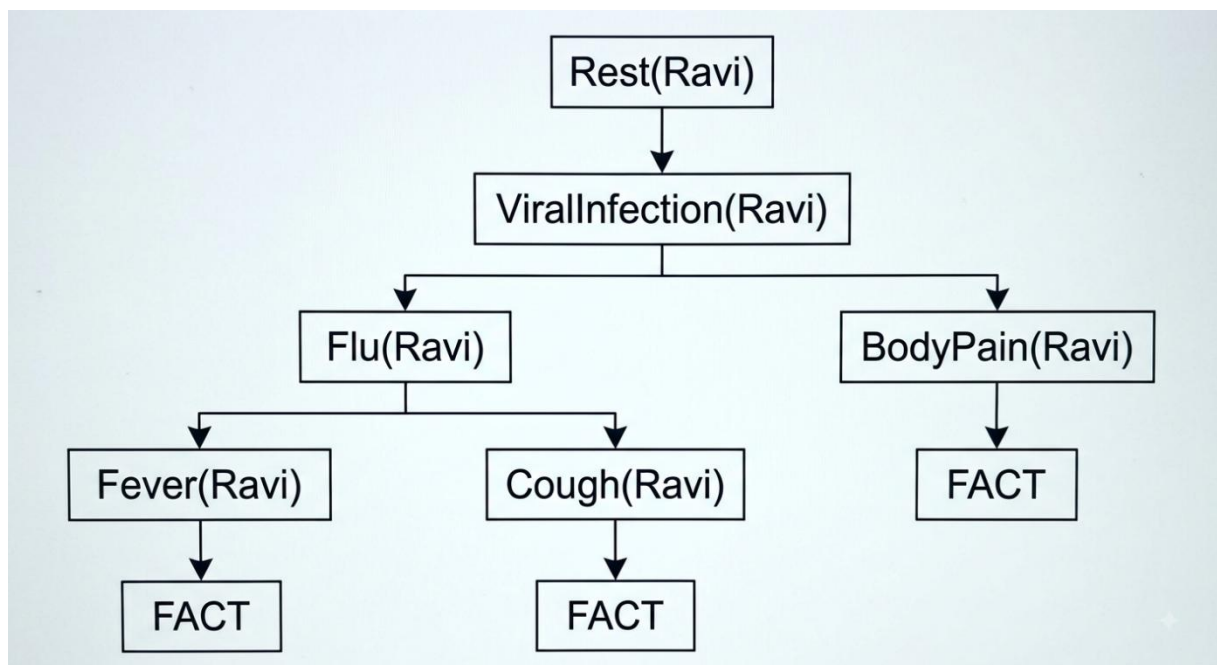


Figure 1: Backward Chaining Reasoning Tree

All required sub-goals are satisfied.

Backward Chaining Result

Rest(Ravi) is successfully proved.

Therefore:

Ravi should be advised to rest.

14. RESOLUTION

Resolution is an inference technique used to prove a query by contradiction. The selected query is:

Query:

$\text{ViralInfection}(\text{Ravi})$

Negated Query:

$\neg \text{ViralInfection}(\text{Ravi})$

To prove this query, the relevant rules are converted into clause form.

14.1 Convert Rule R1 to Clause Form

Original:

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

Equivalent clause:

$\neg \text{Fever}(x) \vee \neg \text{Cough}(x) \vee \text{Flu}(x)$

For Ravi:

$\neg \text{Fever}(\text{Ravi}) \vee \neg \text{Cough}(\text{Ravi}) \vee \text{Flu}(\text{Ravi})$

Given:

$\text{Fever}(\text{Ravi})$

Resolving gives:

$\neg \text{Cough}(\text{Ravi}) \vee \text{Flu}(\text{Ravi})$

Given:

$\text{Cough}(\text{Ravi})$

Resolving again gives:

$\text{Flu}(\text{Ravi})$

Therefore, $\text{Flu}(\text{Ravi})$ is proved.

14.2 Convert Rule R2 to Clause Form

Original:

$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$

Equivalent clause:

$$\neg \text{Flu}(x) \vee \neg \text{BodyPain}(x) \vee \text{ViralInfection}(x)$$

For Ravi:

$$\neg \text{Flu}(\text{Ravi}) \vee \neg \text{BodyPain}(\text{Ravi}) \vee \text{ViralInfection}(\text{Ravi})$$

We have already proved:

$$\text{Flu}(\text{Ravi})$$

and we have the fact:

$$\text{BodyPain}(\text{Ravi})$$

Resolving first with $\text{Flu}(\text{Ravi})$ gives:

$$\neg \text{BodyPain}(\text{Ravi}) \vee \text{ViralInfection}(\text{Ravi})$$

Resolving with:

$$\text{BodyPain}(\text{Ravi})$$

gives:

$$\text{ViralInfection}(\text{Ravi})$$

Therefore, $\text{ViralInfection}(\text{Ravi})$ is derived from Rule R2.

14.3 Resolution by Contradiction

Query:

$$\text{ViralInfection}(\text{Ravi})$$

Negated Query:

$$\neg \text{ViralInfection}(\text{Ravi})$$

Resolve:

$$\text{ViralInfection}(\text{Ravi})$$
$$\neg \text{ViralInfection}(\text{Ravi})$$

□

Since the resolution process produces the empty clause (□), the negation of the query leads to a contradiction. Therefore, the original query $\text{ViralInfection}(\text{Ravi})$ is logically proved.

15. PROTOTYPE IMPLEMENTATION AND RESULTS

15.1 Implementation Overview

A simple Python prototype was developed to demonstrate the rule-based reasoning process of the Smart Healthcare Diagnosis System. The prototype stores the patient's symptoms as facts and applies the three rules defined in the knowledge base.

The inference process follows:

```
Fever(Ravi) + Cough(Ravi)
  ↓ R1
  Flu(Ravi)
  ↓ R2
ViralInfection(Ravi)
  ↓ R3
  Rest(Ravi)
```

15.2 Python Implementation

```
# Initial facts
facts = {
    "Fever(Ravi)",
    "Cough(Ravi)",
    "BodyPain(Ravi)"
}

print("SMART HEALTHCARE DIAGNOSIS SYSTEM")
print("-----")

print("\nInitial Facts:")
for fact in facts:
    print("-", fact)

# Rule R1:
# Fever(x) AND Cough(x) -> Flu(x)

if "Fever(Ravi)" in facts and "Cough(Ravi)" in facts:
    facts.add("Flu(Ravi)")
    print("\nRule R1 Fired:")
    print("Fever(Ravi) AND Cough(Ravi) -> Flu(Ravi)")

# Rule R2:
# Flu(x) AND BodyPain(x) -> ViralInfection(x)

if "Flu(Ravi)" in facts and "BodyPain(Ravi)" in facts:
    facts.add("ViralInfection(Ravi)")
    print("\nRule R2 Fired:")
    print("Flu(Ravi) AND BodyPain(Ravi) -> ViralInfection(Ravi)")

# Rule R3:
# ViralInfection(x) -> Rest(x)

if "ViralInfection(Ravi)" in facts:
```

```

facts.add("Rest(Ravi)")
print("\nRule R3 Fired:")
print("ViralInfection(Ravi) -> Rest(Ravi)")

# Final result
print("\nFinal Inference:")
print("Flu(Ravi) =", "Flu(Ravi)" in facts)
print("ViralInfection(Ravi) =", "ViralInfection(Ravi)" in facts)
print("Rest(Ravi) =", "Rest(Ravi)" in facts)

print("\nFinal Conclusion:")
print("Ravi may have a viral infection and should be advised to rest.")

```

15.3 Sample Output

The following output was obtained by executing the Python prototype:

```

SMART HEALTHCARE DIAGNOSIS SYSTEM
-----

Initial Facts:
- Cough(Ravi)
- BodyPain(Ravi)
- Fever(Ravi)

Rule R1 Fired:
Fever(Ravi) AND Cough(Ravi) -> Flu(Ravi)

Rule R2 Fired:
Flu(Ravi) AND BodyPain(Ravi) -> ViralInfection(Ravi)

Rule R3 Fired:
ViralInfection(Ravi) -> Rest(Ravi)

Final Inference:
Flu(Ravi) = True
ViralInfection(Ravi) = True
Rest(Ravi) = True

Final Conclusion:
Ravi may have a viral infection and should be advised to rest.

```

Figure 2: Sample Output of the Python Prototype

The sample output demonstrates that the prototype successfully applies R1, R2, and R3 in sequence and derives Flu(Ravi), ViralInfection(Ravi), and Rest(Ravi) from the initial facts.

15.4 Implementation Results

Step	Rule Applied	New Fact	Result
1	R1: Fever + Cough → Flu	Flu(Ravi)	PASS
2	R2: Flu + Body Pain → Viral Infection	ViralInfection(Ravi)	PASS
3	R3: Viral Infection → Rest	Rest(Ravi)	PASS

The prototype successfully derives all expected conclusions from the initial facts.

15.5 Discussion of Results

The implementation confirms the forward-chaining inference described in the earlier sections of the report.

The system starts with the three known facts: Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Rule R1 derives:

Flu(Ravi)

Rule R2 then derives:

ViralInfection(Ravi)

Finally, Rule R3 derives:

Rest(Ravi)

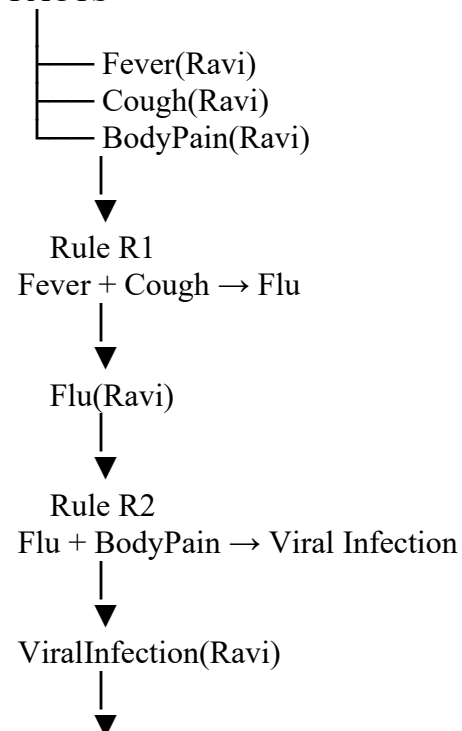
15.6 Prototype Limitation

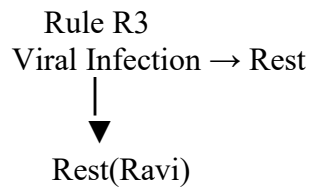
The prototype is a simple proof-of-concept implementation. It uses a small fixed knowledge base and demonstrates forward chaining for the given Ravi example. It is not intended to perform real clinical diagnosis and does not replace professional medical judgment.

16. COMPLETE INFERENCE CHAIN

The complete reasoning process can be represented as:

FACTS





This represents the complete chain of logical inference.

17. FINAL INFERENCE

The system starts with three known facts:

1. $\text{Fever}(\text{Ravi})$
2. $\text{Cough}(\text{Ravi})$
3. $\text{BodyPain}(\text{Ravi})$

Applying Rule R1:

$\text{Fever}(\text{Ravi}) \wedge \text{Cough}(\text{Ravi}) \rightarrow \text{Flu}(\text{Ravi})$

Therefore:

Ravi has flu.

Applying Rule R2:

$\text{Flu}(\text{Ravi}) \wedge \text{BodyPain}(\text{Ravi}) \rightarrow \text{ViralInfection}(\text{Ravi})$

Therefore:

Ravi may have a viral infection.

Applying Rule R3:

$\text{ViralInfection}(\text{Ravi}) \rightarrow \text{Rest}(\text{Ravi})$

Therefore:

Ravi should be advised to rest.

Final Result

The Smart Healthcare Diagnosis System infers that Ravi may have a viral infection and recommends that Ravi should be advised to rest.

This conclusion follows directly from the facts and rules in the knowledge base.

18. FORWARD CHAINING VS BACKWARD CHAINING

Feature	Forward Chaining	Backward Chaining
Approach	Data-driven	Goal-driven
Starting point	Known facts	Query/goal
Direction	Facts → Conclusion	Goal → Required facts
Main purpose	Discover possible conclusions	Prove a specific conclusion
In this system	Derives Flu, Viral Infection and Rest	Proves whether Ravi needs Rest
Suitable use	When many facts are available	When a particular diagnosis/query is required

For this healthcare system, both methods are useful. **Forward chaining** is useful for deriving all possible conditions from observed symptoms, while **backward chaining** is useful when the doctor or system asks a specific question such as whether Ravi needs rest.

19. ROLE OF UNIFICATION AND RESOLUTION

Unification

Unification allows variables in general rules to match specific patients.

For example:

Fever(x) + Fever(Ravi)

gives:

{x/Ravi}

This allows the general rule to be applied specifically to Ravi.

Resolution

Resolution combines clauses to derive new conclusions logically.

For example:

Fever(Ravi)

Cough(Ravi)

and:

$\neg \text{Fever(Ravi)} \vee \neg \text{Cough(Ravi)} \vee \text{Flu(Ravi)}$

lead to:

Flu(Ravi)

Thus, unification helps **match facts with rules**, while resolution helps **derive logically valid conclusions**.

Therefore, unification enables the general rules to be applied to specific facts, while resolution provides a formal method for deriving and validating conclusions.

20. ADVANTAGES OF THE PROPOSED SYSTEM

20.1 Explainable Reasoning

The system can show how a conclusion was reached through a sequence of facts and rules.

20.2 Consistent Decision Making

The same rules can be applied consistently to the available facts.

20.3 Easy Knowledge Representation

Medical knowledge can be represented using simple facts and IF–THEN rules.

20.4 Efficient Inference

Forward and backward chaining can derive conclusions without manually examining every possible relationship.

20.5 Knowledge Updating

New rules and facts can be added to the knowledge base without redesigning the entire reasoning mechanism.

21. LIMITATIONS

1. The system only reasons using the facts and rules available in its knowledge base.
2. The given system is a simplified model and does not represent the complexity of real medical diagnosis.
3. The rules may produce only preliminary conclusions.
4. Real healthcare systems require much larger and validated medical knowledge bases.
5. The system does not replace professional medical diagnosis.
6. Incorrect or incomplete facts can lead to incorrect conclusions.

Therefore, the proposed system should be considered a **knowledge-representation and reasoning model**, rather than a complete clinical diagnostic system.

22. DISCUSSION

The Smart Healthcare Diagnosis System demonstrates the fundamental concepts of Artificial Intelligence and Expert Systems.

The system converts real-world observations into structured logical facts. General medical relationships are represented as rules. The inference engine then applies unification and reasoning techniques to derive new information.

The forward-chaining process starts from Ravi's symptoms and derives possible conditions. In contrast, backward chaining starts with the desired conclusion, such as Rest(Ravi), and determines whether the required conditions can be established.

Resolution provides another formal mechanism for proving conclusions from logical clauses.

Thus, the combination of **knowledge representation and inference techniques** allows the system to transform simple observations into a logically supported recommendation.

23. RESULT

The following results are obtained:

Input / Inference	Result
Ravi has fever	True
Ravi has cough	True
Ravi has body pain	True
Fever + cough	Flu inferred
Flu + body pain	Viral infection inferred
Viral infection	Rest recommended
Query: Rest(Ravi)	Proved
Query: ViralInfection(Ravi)	Proved

Overall Result

Ravi is inferred to have a possible viral infection, and the system recommends rest.

24. CONCLUSION

This report presented the design of a Smart Healthcare Diagnosis System using Artificial Intelligence and Expert System concepts.

The given healthcare knowledge was represented using both **Propositional Logic and First-Order Logic**. A structured knowledge base containing facts and rules was developed. Unification was demonstrated using patient predicates, and the Most General Unifier was identified.

Forward chaining successfully derived:

Fever + Cough $\rightarrow$ Flu

Flu + Body Pain $\rightarrow$ Viral Infection

Viral Infection $\rightarrow$ Rest

Backward chaining successfully proved the query:

Rest(Ravi)

Resolution was applied to prove the selected query ViralInfection(Ravi) by contradiction. The recommendation to rest was then derived using Rule R3.

The final inference is that **Ravi may have a viral infection and should be advised to rest.**

The project demonstrates how an AIES knowledge-based system can represent domain knowledge and perform logical reasoning to produce explainable conclusions.

APPENDIX A – QUICK SUMMARY OF LOGICAL REPRESENTATION

Facts

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Rules

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$

$\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

Final Inference

$\text{Fever}(\text{Ravi}) + \text{Cough}(\text{Ravi})$

↓

$\text{Flu}(\text{Ravi})$

↓

$\text{Flu}(\text{Ravi}) + \text{BodyPain}(\text{Ravi})$

↓

$\text{ViralInfection}(\text{Ravi})$

↓

$\text{Rest}(\text{Ravi})$

Final Answer

Ravi may have a viral infection and should be advised to rest.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.